# TUGAS BESAR 2 IF3170 INTELEGENSI BUATAN
# SEMESTER II TAHUN 2025/2026

## IMPLEMENTASI ALGORITMA PEMBELAJARAN MESIN

**Kelompok 16 - LimaSerangkAI**


**Disusun oleh:**

| | |
|---|---|
| Muhammad Aufa Farabi | 13523023 |
| Joel Hotlan Haris Siahaan | 13523025 |
| Julius Arthur | 13523030 |
| Ferdinand Gabe Tua Sinaga | 13523051 |
| Muhammad Iqbal Haidar | 13523111 |

**PROGRAM STUDI TEKNIK INFORMATIKA**
**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA**
**INSTITUT TEKNOLOGI BANDUNG**

**2025**

# DAFTAR ISI

**PENJELASAN**.....**3**
1. Implementasi Decision Tree Learning..... 3
2. Implementasi Logistic Regression.....4
3. Implementasi SVM.....6
4. Tahap Cleaning dan Preprocessing.....8

**PERBANDINGAN**..... **10**
**PEMBAGIAN TUGAS**..... **11**
**REFERENSI**..... **11**
**LAMPIRAN**.....**11**

# PENJELASAN

## 1. Implementasi Decision Tree Learning

| Kode |
| --- |
| `class TreeNode:`<br>`  def __init__(self, is_leaf, classification = None, attribute = None, threshold = None, branches = None, len_instances = 0):`<br>`  self.is_leaf = is_leaf`<br>`  self.classification = classification`<br>`  self.attribute = attribute`<br>`  self.threshold = threshold`<br>`  self.branches = branches if branches is not None else {}`<br>`  self.len_instances = len_instances` |
| `class Rule:`<br>`  def __init__(self, conditions, classification, accuracy=0.0, y_val_covered=0):`<br>`  self.conditions = conditions # list (attribute, operator, value)`<br>`  self.classification = classification`<br>`  self.accuracy = accuracy`<br>`  self.y_val_covered = y_val_covered #instance dari dataset validasi`<br><br>`  # cek instance validasi masuk ke dalam rule`<br>`  def is_matched(self, instance):`<br>`  for attribute, operator, value in self.conditions:`<br>`  # atribut numerikal -> value merupakan threshold`<br>`  if operator == '<=' and instance[attribute] > value:`<br>`  return False`<br>`  elif operator == '>' and instance[attribute] <= value:`<br>`  return False`<br>`  elif operator == '==' and instance[attribute] != value: # atribut kategorikal`<br>`  return False`<br>`  return True` |
| `class DecisionTreeLearning:` |

```
    def __init__(self, pruning='none'):
        self.tree = None
        self.classes = None
        self.default_class = None
        self.pruning = pruning
        self.n_features = 0
        self.rules = [] # Buat rule post pruning
```

Implementasi algoritma Decision Tree Learning (DTL) mengacu pada algoritma C4.5. Proses pembentukan pohon dilakukan secara rekursif dengan memilih atribut percabangan terbaik berdasarkan Gain Ratio untuk mengurangi bias pada atribut dengan kardinalitas tinggi. Algoritma ini menangani data bertipe numerik dengan mencari best threshold yang memisahkan data secara optimal. Selain itu, juga diimplementasikan fitur Post-Pruning dengan strategi Reduced Error Pruning dan Rule Pruning guna mencegah overfitting dan meningkatkan kemampuan generalisasi model.

## 2. Implementasi Logistic Regression

Kode

```
class MinibatchGradientDescent:
    def __init__(self, lr=0.1, epochs=10, batch_size=32):
        self.lr = lr
        self.epochs = epochs
        self.batch_size = batch_size
        self.weights = None
        self.bias = None

    def sigmoid(self, z):
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0

        for epoch in range(self.epochs):
            idx = np.random.permutation(n_samples)
            X_shuffled = X[idx]
            y_shuffled = y[idx]
```

```
            for i in range(0, n_samples, self.batch_size):
                batch_end = min(i + self.batch_size, n_samples)
                X_batch = X_shuffled[i:batch_end]
                y_batch = y_shuffled[i:batch_end]

                linear_model = np.dot(X_batch, self.weights) + self.bias
                y_predicted = self.sigmoid(linear_model)

                error = y_batch - y_predicted

                self.weights += self.lr * np.dot(X_batch.T, error) /
len(X_batch)
                self.bias += self.lr * np.mean(error)

    def predict(self, X):
        linear = np.dot(X, self.weights) + self.bias
        pred = self.sigmoid(linear)
        return np.where(pred >= 0.5, 1, 0)

    def compute_loss(self, X, y):
        n_samples = X.shape[0]
        linear_model = np.dot(X, self.weights) + self.bias
        y_pred = self.sigmoid(linear_model)

        epsilon = 1e-15
        y_pred = np.clip(y_pred, epsilon, 1 - epsilon)

        loss = -np.mean(y * np.log(y_pred) + (1 - y) * np.log(1 -
y_pred))
        return loss
```

```
class LogisticRegressionOVO:
    def __init__(self, lr = 0.1, epochs = 10, batch_size=32):
        self.lr = lr
        self.epochs = epochs
        self.batch_size = batch_size
        self.models = None
        self.n_classes = None
        self.label_to_idx = None
```

```
        self.idx_to_label = None
```

Implementasi algoritma logistic regression menggunakan Mini-batch gradient descent. Pada proses fit() update bobot dilakukan berdasarkan sekelompok kecil sampel (batch) dengan rumus weights += lr * error * xi. Fungsi sigmoid() digunakan untuk memetakan hasil linear ke probabilitas antara 0 dan 1 dengan rumus 1 / (1 + exp(-z)). Algoritma ini juga mengimplementasikan strategi One vs one untuk data multiclass sehingga prediksi akhir diambil berdasarkan voting terbanyak.

### 3. Implementasi SVM

Kode

```
#Algoritma Optimasi SMO oleh JOHN PLATT
class SVM_Binary:
   def __init__(self, data_train: pd.DataFrame, y_train: pd.Series,
penalty: float = 1.0, kernel: str = 'rbf') -> None:
        self.data = np.asarray(data_train, dtype=float)
        self.y = np.asarray(y_train, dtype=float)

        self.C = penalty
        self.max_passes = 5
        self.eps = 1e-8

        if kernel == 'rbf':
            self.gamma = 1 / self.data.shape[1]
            self.kernel = self.rbf_kernel
        elif kernel == 'linear':
            self.kernel = self.linear_kernel
        else:
            raise ValueError("Kernel tidak dikenal")

        self.b = 0.0
        self.alpha = np.zeros(len(self.data))
        self.E = -self.y.copy()
        self.cache_kernel()
```

```
#Algoritma SVM dengan cvxopt
class SVM_Binary_cvxopt:
   def __init__(self, data_train, y_train, penalty=1.0,
kernel='rbf'):
```

```
        self.data = np.asarray(data_train, dtype=float)
        self.y = np.asarray(y_train, dtype=float)
        self.C = penalty

        self.gamma = 1 / self.data.shape[1]

        self.kernel = self.rbf_kernel if kernel == 'rbf' else
self.linear_kernel
        self.cache_kernel()
        self.eps = 1e-8


def linear_kernel(self, x, z):
        return np.dot(x, z)

    def rbf_kernel(self, x, z):
        return np.exp(-self.gamma * np.linalg.norm(x - z)**2)

    def cache_kernel(self):
        n = len(self.data)
        K = np.zeros((n, n))
        for i in range(n):
            for j in range(n):
                K[i, j] = self.kernel(self.data[i], self.data[j])
        self.kernel_matriks = K

    def fit_binary(self):

        P = matrix(np.outer(self.y, self.y) * self.kernel_matriks)
        q = matrix(-1.0 * np.ones(len(self.data)))

        G = matrix(np.vstack((-np.eye(len(self.data)),
                              np.eye(len(self.data)))))
        h = matrix(np.hstack((np.zeros(len(self.data)),
                              np.ones(len(self.data)) * self.C)))

        A = matrix(self.y.reshape(1, -1))
        b = matrix(0.0)
```

```
        solution = solvers.qp(P, q, G, h, A, b)
        alphas = np.array(solution['x']).flatten()

        idx = alphas > self.eps
        self.support_alphas = alphas[idx]
        self.support_vectors = self.data[idx]
        self.support_labels = self.y[idx]

        b_list = []
        for i, xi in enumerate(self.support_vectors):
            k = np.array([self.kernel(xi, sv) for sv in
self.support_vectors])
            b_i = self.support_labels[i] -
np.sum(self.support_alphas * self.support_labels * k)
            b_list.append(b_i)

        self.b = np.mean(b_list)

        return alphas


    def predict(self, X_test):
        X_test = np.asarray(X_test, dtype=float)
        result = []

        for x in X_test:
            k = np.array([self.kernel(x, sv) for sv in
self.support_vectors])
            fx = np.sum(self.support_alphas * self.support_labels
* k) + self.b
            result.append(np.sign(fx))

        return np.array(result)
```

```
class SVM:
   def __init__(self, data_train: pd.DataFrame, y_train: pd.Series,
penalty: float = 1.0, kernel: str = 'rbf', smo =True) -> None:
```

```
        self.models = [] #berisi instance dari SVM_binary
        self.raw_data = data_train
        self.raw_y = y_train

        self.C = penalty
        self.kernel = kernel
        self.smo =smo

    def filter_data(self, labels1, labels2):
        filtered_data = self.raw_data[(self.raw_y == labels1) |
(self.raw_y == labels2)].copy()
        filtered_y = self.raw_y[(self.raw_y == labels1) | (self.raw_y ==
labels2)].copy()

        filtered_y[filtered_y == labels1] = 1
        filtered_y[filtered_y == labels2] = -1

        print('Ini len raw ' + str(len(filtered_data)))
        print('Ini len y ' + str(len(filtered_y)))

        return filtered_data, filtered_y


    def fit(self):
        unique_labels = np.unique(self.raw_y)
        count_unique = len(unique_labels)

        for i in range(count_unique):
            for j in range(i + 1, count_unique):
                label_a = unique_labels[i]
                label_b = unique_labels[j]

                filtered_data, filtered_y =
self.filter_data(unique_labels[i], unique_labels[j])
                model= None

                model = SVM_Binary(filtered_data, filtered_y, self.C,
self.kernel)

                model.fit_binary()
```

```
                self.models.append([label_a, label_b, model])

    def predict(self, X_predict):
        X_arr = np.asarray(X_predict, dtype=float)
        if X_arr.ndim == 1:
            X_arr = X_arr.reshape(1, -1)

        predictions = []
        for i in range(len(X_arr)):
            x = X_arr[i]
            voting = {} #berisi {key:count}

            for (label_a, label_b, svm_model) in self.models:
                pred = svm_model.predict([x])[0]
                if pred == 1:
                    key = label_a
                else:
                    key = label_b

                voting[key] = voting.get(key, 0) + 1

            max_vote = max(voting.values())
            candidates = [label for label, v in voting.items() if v ==
max_vote]
            final_label = min(candidates)
            predictions.append(final_label)

        return predictions
```

Algoritma SVM ini mengimplementasikan Sequential Minimal Optimization by John Platt yang memecah permasalahan menjadi sub-masalah kecil sehingga hanya mengoptimalkan 2 Lagrange multipliers atau alpha pada satu waktu. Kernel yang digunakan adalah Radial Basis Function (RBF) yakni np.exp(-self.gamma * np.linalg.norm(x - z)**2) yang lebih akurat untuk data kompleks ketimbang kernel linear. Algoritma ini juga mengimplementasikan strategi One vs one untuk data multiclass sehingga prediksi akhir diambil berdasarkan voting terbanyak. Selain itu juga dilakukan caching kernel untuk mengoptimasi penggunaan memori dan mempersingkat waktu eksekusi.

## 4. Tahap Cleaning dan Preprocessing

Pada tahapan ini data melewati 5 proses antara lain adalah:

1. Penanganan missing value menggunakan imputasi median untuk data bertipe numerik dan modus untuk data bertipe kategorikal. Alasan memilih menggunakan imputasi median untuk data bertipe numerik karena lebih tahan terhadap data outlier/skewed ketimbang menggunakan imputasi mean/rata-rata. Sedangkan untuk data bertipe kategorikal menggunakan imputasi modus karena itu yang biasa digunakan pada tipe data tersebut.
2. Penanganan data duplikat dilakukan dengan membuang data duplikat dari dataset. Alasan membuang data duplikat adalah untuk meningkatkan efisiensi pada saat proses training dan mengurangi risiko model terhadap bias.
3. Melakukan feature engineering berupa transformasi logaritma pada fitur yang skewed serta membuat fitur turunan baru berupa Approval_Rate untuk menangkap pola rasio kelulusan SKS mahasiswa. Tujuannya adalah untuk menormalkan distribusi data yang skewed dan menciptakan fitur baru yang lebih relevan untuk training model.
4. Scaling data menggunakan RobustScaler. Alasan melakukannya karena algoritma Logistic Regression dan SVM sensitif terhadap skala data, sehingga diperlukan scaling agar tidak terjadi dominansi fitur dan untuk mempercepat algoritma mencapai konvergen. RobustScaler digunakan karena metode ini menggunakan Interquartile Range (IQR) yang lebih tahan (robust) terhadap outlier dibandingkan StandardScaler
5. Encoding data menggunakan kombinasi OneHotEncoder untuk fitur kardinalitas rendah dan TargetFeatureEncoder untuk fitur kardinalitas tinggi. TargetFeatureEncoder mengubah data kategorikal menjadi nilai rata-rata dari variabel target untuk kategori tersebut. Alasan penggunaannya adalah karena Logistic Regression dan SVM membutuhkan fitur bertipe numerikal untuk melatih model dan melakukan prediksi.
6. Penanganan imbalanced data menggunakan SMOTE. Alasan dilakukan ini adalah untuk menyeimbangkan jumlah sampel data training untuk setiap kelas target supaya model tidak bias terhadap kelas mayoritas.
7. Feature Selection menggunakan SelectKBest dengan pemilihan 5 fitur yang paling berpengaruh terhadap nilai target. Alasan melakukan ini untuk meningkatkan efisiensi komputasi saat proses training dan mengurangi fitur yang kemungkinan besar hanya berupa noise.

# PERBANDINGAN

| Algoritma | Skor Accuracy From Scratch | Skor Accuracy scikit-learn | Analisis |
|---|---|---|---|
| Decision Tree Learning (C4.5) | 0.7299412915851272 | 0.7299412915851272 | Hasil eksperimen menunjukkan bahwa implementasi from scratch kami mampu bersaing menyamai performa dengan library standar pada scikit-learn untuk dataset ini. Keunggulan ini kemungkinan besar disebabkan oleh penerapan teknik Reduced Error Pruning yang agresif dan spesifik, serta pemilihan fitur (Feature Selection) yang tepat. Hal ini mengindikasikan bahwa logika algoritma Gain Ratio dan mekanisme pemangkasan pohon (pruning) yang kami bangun telah bekerja secara efektif untuk mencegah overfitting dan meningkatkan generalisasi model. |
| Logistic Regression | 0.735812133072407 | 0.636986301369863 | Hasil eksperimen menunjukkan bahwa implementasi SGD manual yang didukung oleh preprocessing yang tepat mampu bersaing dengan performa library scikit-learn. Strategi One-vs-One terbukti efektif untuk menangani klasifikasi multiclass, meskipun dengan kompensasi waktu training yang lebih lama karena harus melatih sejumlah model biner. |
| SVM (John Platt) | 0.687866927592955 | 0.687866927592955 | Hasil eksperimen menunjukkan bahwa kesamaan hasil akurasi hingga tingkat desimal ini menandakan implementasi scratch kami telah berhasil mereplikasi logika matematis SVM secara presisi. Algoritma SMO yang kami bangun untuk menyelesaikan masalah optimasi kuadratik, beserta implementasi kernel RBF dan strategi margin maximization, terbukti bekerja setara dengan yang ada pada scikit-learn untuk konfigurasi parameter yang sama (C=1.0, gamma='scale'). |

| | | | |
|---|---|---|---|
| SVM (cvxopt) | 0.5225048923679061 | 0.687866927592955 | Hasil eksperimen menunjukkan bahwa terdapat perbedaan yang cukup signifikan antara metode-metode pelatihan SVM di atas. Perbedaan tersebut dapat disebabkan oleh beberapa faktor, seperti perbedaan metode optimasi yang digunakan serta stabilitas numeriknya. SVM yang dilatih menggunakan library CVXOPT memanfaatkan metode Quadratic Programming berbasis Interior-Point Method (IPM), yang sangat sensitif terhadap kondisi matriks dan parameter kernel yang digunakan. |

## PEMBAGIAN TUGAS

| NIM | Tugas |
|---|---|
| 13523023 | Implementasi Decision Tree Learning |
| 13523025 | Implementasi Logistic Regression |
| 13523030 | Data cleaning dan preprocessing |
| 13523051 | Implementasi SVM |
| 13523111 | Laporan |

## REFERENSI


Platt, J. C. (1998). Sequential Minimal Optimization: A Fast Algorithm for Training Support Vector Machines. Retrieved December 4, 2025, from https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/tr-98-14.pdf

Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.


## LAMPIRAN

- Link Google Collab
- Link Repository Github